

IOCTeaper – Application Design Specification

Manudeep Maddipatla – mmaddipa@umd.edu

121417350 – mmaddipa

Table of Contents

Introduction	5
1.1 Purpose.....	5
1.2 Scope	5
2. Application Overview.....	5
2.1 Technology Stack.....	5
2.2 Overall architecture.....	5
3. System Architecture	6
4. Application Software Components	7
4.1 IOCParse.....	7
4.2 FangerDefanger	7
4.3 Categorizer	7
4.4 TagManager	7
4.5 Searcher.....	8
4.6 Exporter	8
4.7 SessionManager	8
5. Application UI Components.....	8
5.1 Home Page	8
5.2 Results Page	9
5.3 Tagging Panel.....	9
5.4 Search Bar.....	9
5.5 Export Panel	9
5.6 Clear Session Button.....	10
6. Application Data Components	10
6.1 IOC Object	10
6.2 IOC Store.....	11
6.3 Tag Store	11
6.4 Data Flow	11
6.5 Export and Copying Formats	11
7. Test Cases	11
7.1 FR-001: Parse IOCs from Raw Text	11
7.2 FR-002: Categorize Extracted IOCs	11
7.3 FR-003: Apply Filters and Normalization	12
7.4 FR-004: Search Results	12

7.5 FR-005: Copy to Clipboard	12
7.6 FR-006: Export Results to File	12
7.7 FR-007: Clear Session	12
7.8 FR-008: Session-Scoped Data Handling	12
7.9 FR-009: Input Validation & Size Limits.....	12
7.10 FR-010: Error Messaging	13
7.11 FR-011: Add Tags to IOCs	13
7.12 FR-012: Manage Tags.....	13
7.13 FR-013: Export Tags with IOCs	13
8. Traceability Matrix.....	13
References	14

Table of Figures

Figure 1. Data Flow between Components.....	6
Figure 2. Application UI Components Interaction	10

Introduction

1.1 Purpose

The purpose of this document is to describe the design details of the IOC extractor application IOCREAPER. Specifically this document provides a detailed execution plan for developing the IOCREAPER application [1] [2]

1.2 Scope

This document achieves the above purpose by defining various architectural, interface and core components of the application and provides a clear mapping to design elements.

It also delves into the UI design elements, data models relevant to the application and finally concludes by defining and explaining the test use case to ensure that the application performs as intended.

2. Application Overview

IOCREAPER is a browser based, locally run web application which is designed to extract, modify, search and tag specific data patterns called as Indicators of Compromise from a raw text input. The application in its current iteration does not have any external API or persistent database access to ensure security, privacy, and co-alignment with secure coding practices.

2.1 Technology Stack

Designing this application will be primarily done using Python with FastAPI due to its support for quick prototyping and input validation.

The below section further elaborates on the complete details of the stack that will be used:

- Python → core programming language
- FastAPI → For web framework [3]
- HTML/CSS/Bootstrap → For frontend styling and layout (progressive enhancement)
- JavaScript → For clipboard copying, tagging, and search interactions.
- Storage → In-memory storage (Python dictionaries/lists for session data).

2.2 Overall architecture

IOCREAPER follows a layered architecture approach with clear distinction for concerns and functionality:

1. Web Layer: This layer manages all the HTTP requests and responses.

2. UI Layer: This layer provides a browser-based interface for input, results display, and user actions such as tagging, searching etc.
3. Application Logic Layer: This layer is where the implementation logic parsing, categorization, searching, tagging, and exporting can be found.
4. Data Layer: This layer stores IOC objects and tags in memory during a user session.

The goal for this design approach is to ensure modularity, maintainability, and secure operation without persistence.

3. System Architecture

This application is based on a client – server model, a broken-down construction of which can be found below.

1. Clients (Security Analysts, Threat Intel Analysts) – Clients interact with the web interface via a browser locally.
2. The server (FastAPI backend) processes requests and coordinates related to IOC extraction, tagging, searching, copying, and exporting.
3. Data flows in memory only, ensuring privacy.

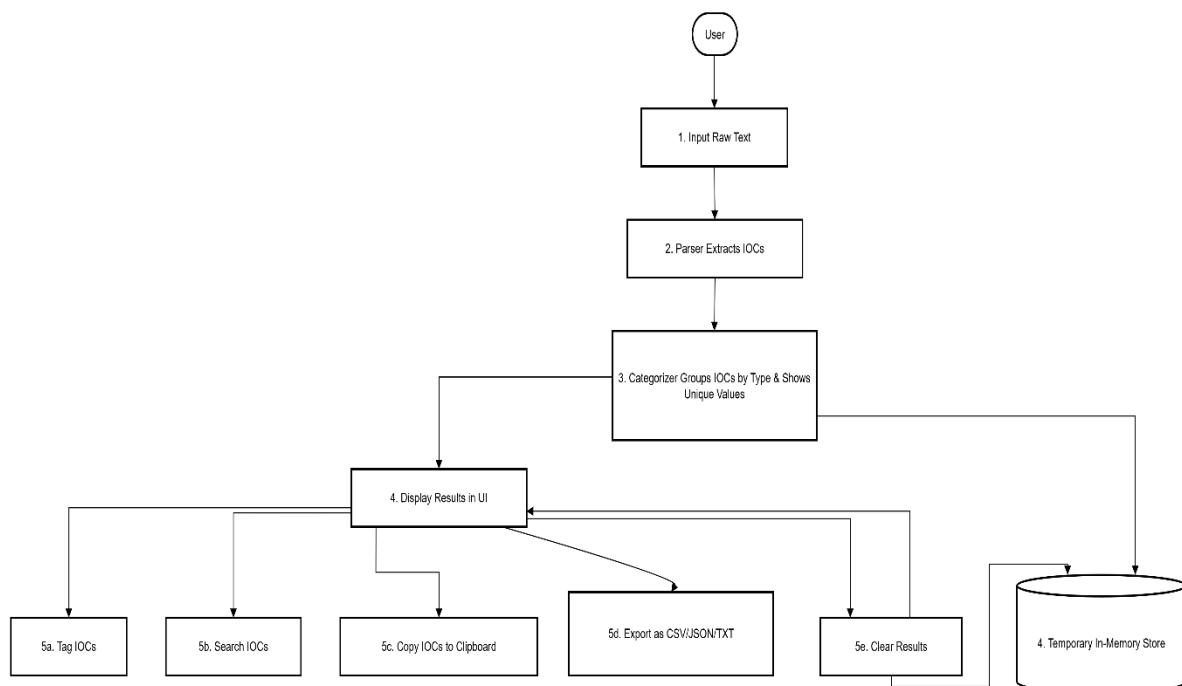


Figure 1. Data Flow between Components

4. Application Software Components

4.1 IOCParser

- Purpose: This component extracts IOCs from raw text using regex or the iocextract library.
- Classes/Models Needed: IOCParser class.
- Relevant Use Cases:
 - Use Case 1: Upload and extract IOCs from raw text.
 - Use Case 7: Handle extraction errors (invalid regex, no IOCs found).
- Core Business Logic Relation: This is the central engine for IOC identification. Other components (Categorizer, Exporter, TagManager) depend on it and this component generates valid input.

4.2 FangerDefanger

- Purpose: This component converts IOCs between defanged IOCs (hxxp://, example[.]com) and fanged versions.
- Classes/Models Needed: FangerDefanger class.
- Relevant Use Cases:
 - Use Case 1: Extraction should normalize IOCs.
- Core Business Logic Relation: This component is responsible for pre-processing step that ensures consistency across parsing, categorization, and exporting.

4.3 Categorizer

- Purpose: This component organizes IOCs into types (IP, Domain, URL, Hash, Email). It only shows unique values.
- Classes/Models Needed: Categorizer class.
- Relevant Use Cases:
 - Use Case 1: After extraction, group IOCs for display.
- Core Business Logic Relation: This component provides a structured organization of extracted data which enables tagging, searching, and exporting.

4.4 TagManager

- Purpose: This component allows users to assign, update, and remove tags to IOCs.
- Classes/Models Needed:
 - TagManager class
- Relevant Use Cases:
 - Use Case 4: Tag IOCs by type.
 - Use Case 8: Export tags with IOCs.
- Core Business Logic Relation: This module extends IOC data with analyst-added context, essential for threat intelligence workflows.

4.5 Searcher

- Purpose: This component is tasked with supporting searching for IOCs and tags for quick filtering and copying.
- Classes/Models Needed: Searcher class.
- Relevant Use Cases:
 - Use Case 3: Search for specific IOCs in uploaded text.
 - Use Case 4 and 8: Search tags.
- Core Business Logic Relation: This component connects user navigation (finding IOCs) with stored data, enhancing usability.

4.6 Exporter

- Purpose: This component exports IOCs into CSV, JSON, or TXT files. Optionally includes tags.
- Classes/Models Needed: Exporter class.
- Relevant Use Cases:
 - Use Case 2: Export extracted IOCs.
 - Use Case 8: Export tags with IOCs.
- Core Business Logic Relation: Provides persistence outside the app session (file-based), critical for analysts to reuse data.

4.7 SessionManager

- Purpose: This module is tasked with handling in-memory IOC storage, session lifecycle, and clearing data.
- Classes/Models Needed: SessionManager class.
- Relevant Use Cases:
 - Use Case 6: Delete text/input from window.
 - Use Case 7: Handle session reset after errors.
- Core Business Logic Relation: This component ensures IOC data is temporary (stateless) by design and prevents data persistence beyond session scope.

5. Application UI Components

The application will have a page like display view with every session having the below types of page

5.1 Home Page

- Elements:
 - A text input box which allows for multiline text to be pasted .
 - A “Extract” button to trigger IOC extraction.
 - There will also be an option checkboxes/toggles to:
 - Normalize IOCs (defang/refang).
 - Deduplicate results.
- Navigation:

- After pressing Extract button, user is redirected to Results Page with extracted IOCs.

5.2 Results Page

— Elements:

- A tabbed or windowed view with clearly demarcated IOC types :
 - Tab 1: IP addresses.
 - Tab 2: Domains.
 - Tab 3: URLs.
 - Tab 4: Hashes.
 - Tab 5: Emails.
- Each table shows details of unique values + counts.

— Navigation:

- Tabs switch between IOC categories using mouse movements or pressing tabs .
- From this page, users can access other pages such as Tagging Panel, Search, Export, or Clear Session.

5.3 Tagging Panel

— Elements:

- A dropdown menu or text input for adding tags (e.g., “phishing”, “C2”, “malware”).
- These tags are displayed (chips/badges) next to IOCs.
- Tags can be removed with a small “x” on each tag.

— Navigation:

- Accessed from Results Page → select IOC(s) → “Tag IOCs” button.
- Returns to Results Page after tagging.

5.4 Search Bar

— Elements:

- An input box at the top of Results Page.
- Search can be conducted in two modes:
 - Search by IOC value.
 - Search by tag.

— Navigation:

- Typing an IOC value filters the displayed table in Results Page.

5.5 Export Panel

— Elements:

- A button and a drop-down menu to export as CSV, JSON and TXT.
- There is an additional option to include tags in export.

— Navigation:

- This action is available on Results Page toolbar.
- Exporting triggers file download to user’s machine.

5.6 Clear Session Button

— Elements:

- A One-click Clear All button.
- This action is accompanied with a warning dialog with a message along the lines of “Are you sure you want to clear all data?”

— Navigation:

- This action is present in Results Page toolbar.
- This will clear UI tables and memory (handled by SessionManager).

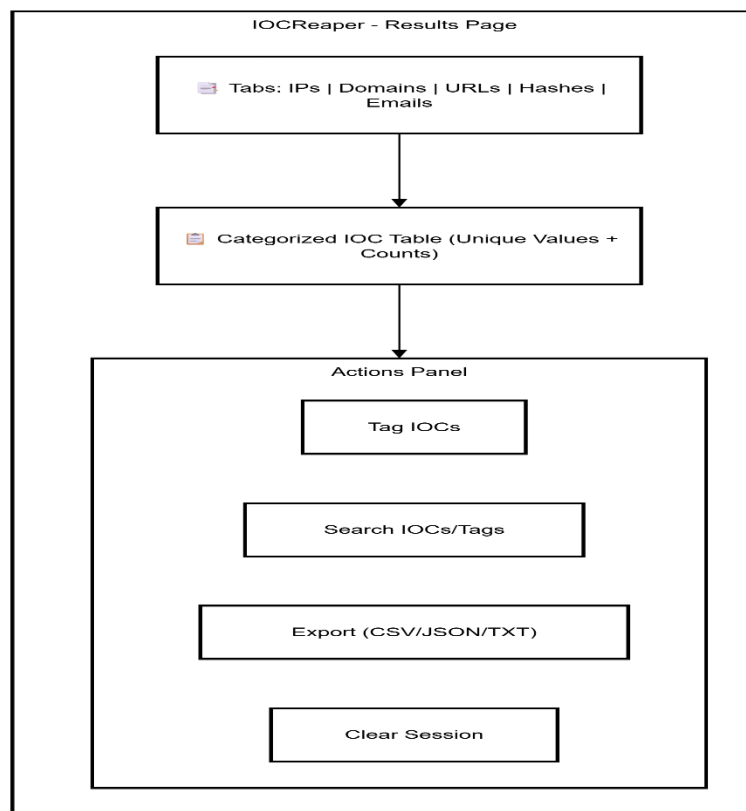


Figure 2. Application UI Components Interaction

6. Application Data Components

The IOCReaper application only uses transient in-memory storage only. No external or persistent database is involved in its working . All the data is cleared when the session ends, or the user resets the application.

6.1 IOC Object

— This object represents a single Indicator of Compromise.

— Fields:

- IOC_value → the IOC string (IP, domain, URL, hash, etc.).
- IOC_type → IOC category (e.g., "IP", "Domain").
- IOC_Tag → list of user-defined labels.

6.2 IOC Store

- This data type is a Dictionary keyed by IOC_type and is mapped to lists of IOC objects.
- It supports categorization, uniqueness checks, and fast lookups.

6.3 Tag Store

- This Dictionary data type which is keyed by IOC_Tag variable and is mapped mapping to IOC objects.
- It enables tag-based searching and exporting IOCs with or without their associated tags.

6.4 Data Flow

- When a user inputs text here , Parser extracts IOCs.
- Categorizer groups them into IOC Store.
- TagManager updates both IOC objects and Tag Store.
- Search and export functions will data pull from these structures.
- Clear session wipes all in-memory data.

6.5 Export and Copying Formats

- The following data formats are supported for copying and exporting data
 - **CSV:** Rows with value, type, tags.
 - **JSON:** Structured objects.
 - **TXT:** Plain text list of IOCs, optionally with tags.
- **Copying :** This action will copy a new line delimited list of IOCs into the clipboard .

7. Test Cases

This section details test cases for each functional requirements and their working and success criteria . [4]

7.1 FR-001: Parse IOCs from Raw Text

- How to Test: User must provide a raw text with multiple IOC types, then click *Extract*.
- What Proves Design Works: Regex-based extraction identifies, and parses valid IOCs and these unique values are displayed in the Results Section .
- Success Criteria: All valid IOCs are displayed, with normalization/deduplication applied if selected.

7.2 FR-002: Categorize Extracted IOCs

- How to Test: User must provide a raw text with multiple IOC types, then click *Extract*.
- What Proves Design Works: IOCs are grouped into categories with unique counts.
- Success Criteria: Each IOC appears in the correct category tab with no duplicates.

7.3 FR-003: Apply Filters and Normalization

- How to Test: Once text is parsed and filters such as deduplication, refang, or normalization are selected .
- What Proves Design Works: Selected transformations are applied correctly.
- Success Criteria: Results reflect chosen filters without invalid transformations.

7.4 FR-004: Search Results

- How to Test: Using the search bar look for specific IOC values or tags.
- What Proves Design Works: Matching results are highlighted and displayed across categories.
- Success Criteria: All matches appear , if none exist “No results found” is displayed .

7.5 FR-005: Copy to Clipboard

- How to Test: Click on *Copy IOCs* or use keyboard shortcut to copy extracted IOCs.
- What Proves Design Works: Selected IOCs are copied into the system clipboard.
- Success Criteria: Pasting shows newline-separated IOC values identical to displayed results.

7.6 FR-006: Export Results to File

- How to Test: Choose *Export* and select CSV, JSON, or TXT.
- What Proves Design Works: File are downloaded successfully with correct format and content.
- Success Criteria: Exported file contains all extracted IOCs, properly structured.

7.7 FR-007: Clear Session

- How to Test: After parsing results, click *Clear Session*.
- What Proves Design Works: In-memory data is erased and UI resets back to its initial state.
- Success Criteria: Input area is blank, and no results remain.

7.8 FR-008: Session-Scoped Data Handling

- How to Test: Open app, parse IOCs, then close and reopen app.
- What Proves Design Works: No data persists between sessions.
- Success Criteria: Each session starts fresh with empty state.

7.9 FR-009: Input Validation & Size Limits

- How to Test: Provide input >100 KB and normal-size input.
- What Proves Design Works: Oversized input is rejected with an error; valid input is parsed.
- Success Criteria: Only supported input size is processed.

7.10 FR-010: Error Messaging

- How to Test: Trigger errors (invalid tag, malformed IOC, oversized input, no results).
- What Proves Design Works: Application displays user-friendly error messages without crashing.
- Success Criteria: Errors are reported clearly; retry option is available.

7.11 FR-011: Add Tags to IOCs

- How to Test: Select IOCs and assign tags via tagging panel.
- What Proves Design Works: Tags are attached and displayed with IOCs.
- Success Criteria: IOCs consistently show correct tag associations.

7.12 FR-012: Manage Tags

- How to Test: Rename, delete, or modify tags from Tag Manager.
- What Proves Design Works: Tag updates reflect across all associated IOCs.
- Success Criteria: Modified or removed tags apply universally; invalid input is rejected.

7.13 FR-013: Export Tags with IOCs

- How to Test: Apply tags to IOCs, then export as CSV/JSON/TXT.
- What Proves Design Works: Exported file includes tag mappings alongside IOCs.
- Success Criteria: Tags appear correctly in exported data; if none exist, export proceeds without tags.

8. Traceability Matrix

Requirement ID	Requirement Description	Implemented By (Design Components)
FR-001	Parse IOCs from raw text	IOCParser, FangerDefanger, UI Layer
FR-002	Categorize extracted IOCs	Categorizer, IOC Store, UI Layer
FR-003	Apply filters and normalization	FangerDefanger, Categorizer, UI Layer
FR-004	Search results	Searcher, Tag Store, UI Layer
FR-005	Copy to clipboard	Exporter (copy method), UI Layer
FR-006	Export results to file	Exporter, TagManager (when including tags), UI Layer
FR-007	Clear session	SessionManager, IOC Store, Tag Store, UI Layer
FR-008	Session-scoped data handling	SessionManager, IOC Store, Tag Store
FR-009	Input validation & size limits	IOCParser (validation), UI Layer

FR-010	Error messaging	IOCParser, SessionManager, UI Layer
FR-011	Add tags to IOCs	TagManager, IOC Store, UI Layer
FR-012	Manage tags	TagManager, Tag Store, UI Layer
FR-013	Export tags with IOCs	Exporter, TagManager, Tag Store

References

- [1] "IEEE Recommended Practice for Software Design Descriptions," *IEEE Std 1016-1998*, pp. 1–23, Dec. 1998, doi: 10.1109/IEEESTD.1998.88828.
 - [2] "IOCTree - Application Description and Use Cases."
 - [3] "FastAPI." Accessed: Sept. 29, 2025. [Online]. Available: <https://fastapi.tiangolo.com/>
 - [4] "IOCTree - Application Requirements Specification."
-